

目录

Tricky	1
左偏树	1
二叉搜索树	2
虚树	4
LinkCutTree	4
点分治	6
线段树合并	8
李超线段树	11
莫队	13
树上莫队	13
带修莫队	16
只增莫队	17

Tricky

1. 线段树分治

左偏树

```
#include<cstdio>
inline void Swap(int &x,int &y){int swap=x;x=y;y=swap;}
struct LeftistTree{
    int Dist[100010],Data[100010],Son[100010][2],Dad[100010];
    inline void Merge(int x,int y,int &root){
        if(Data[x]==-1||Data[y]==-1||x==y)return;
        int fx=FindDad(x);
        int fy=FindDad(y);
        Dad[fx]=Dad[fy]=root=Merge(fx,fy);
    }
    inline int Merge(int x,int y){
        if(x==0||y==0)return x+y;
        if(Data[x]>Data[y]||(Data[x]==Data[y]&& x>y))
            Swap(x,y);
        Son[x][1]=Merge(Son[x][1],y);
        Dad[Son[x][1]]=x;
        if (Dist[Son[x][0]]<Dist[Son[x][1]])
            Swap(Son[x][0],Son[x][1]);
        Dist[x]=Dist[Son[x][1]]+1;
        return x;
    }
    inline int FindDad(int x){
        if(x==Dad[x])return x;
```

```

        else return Dad[x]=FindDad(Dad[x]);
    }
    inline void Pop(int x){
        if(Data[x]==-1)return ;
        x=FindDad(x);
        Data[x]=-1;
        Dad[Son[x][0]]=Son[x][0];
        Dad[Son[x][1]]=Son[x][1];
        Dad[x]=Merge(Son[x][0],Son[x][1]);
    }
    inline int Top(int x){
        if(Data[x]==-1)return -1;
        x=FindDad(x);
        return Data[x];
    }
};LeftistTree T;
int n,m,opt,x,y;
int main(){
    scanf("%d%d",&n,&m);T.Dist[0]=-1;
    for(int i=1;i<=n;i++)
        T.Dad[i]=i,scanf("%d",&T.Data[i]);
    for(int i=1;i<=m;i++){
        scanf("%d%d",&opt,&x);
        if(opt==1){
            scanf("%d",&y);
            T.Merge(x,y,x);
        }
        else{
            printf("%d\n",T.Top(x));
            T.Pop(x);
        }
    }
    return 0;
}

```

二叉搜索树

```

template<typename T>
struct ChxSet{
    typedef __gnu_pbds::tree<T,__gnu_pbds::null_type,std::less<T>,__gnu_pbds::rb_tree_tag,__gnu_pbds::t
    Type S;
    void insert(T x){S.insert(x);}
    bool have(T x){return S.find(x)!=S.end();}
    bool erase(T x){return S.erase(x);}
    typename Type::iterator find(T x){return S.find(x);}
    typename Type::iterator erase(typename Type::iterator it){return S.erase(it);}
    int getrank(T x){return S.order_of_key(x);}//返回严格小于 x 的元素个数 (以 Cmp_Fn 作为比较逻辑), 即
    typename Type::iterator select(int x){return S.find_by_order(x);}//返回 Cmp_Fn 比较的排名所对应元素
    typename Type::iterator prev(T x){//<x 严格前驱
        int t=S.order_of_key(x);
        if(t==0)return S.end();
    }
}

```

```

        else return S.find_by_order(x-1);
    }
    typename Type::iterator succ(T x){///x 严格后继
        return S.upper_bound(x);
    }
    typename Type::iterator lower_bound(T x){return S.lower_bound(x);}
    typename Type::iterator upper_bound(T x){return S.upper_bound(x);}
    typename Type::iterator begin(){return S.begin();}
    typename Type::iterator end(){return S.end();}
    bool empty(){return S.empty();}
    size_t size(){return S.size();}
};

template<typename T,int MaxN>///MaxN: 总共入队次数
struct ChxMultiSet{
    typedef __gnu_pbds::tree<std::pair<T,int>__,__gnu_pbds::null_type,std::less<std::pair<T,int>>__,__gnu_p
    Type S;
    int index;
    void insert(T x){index++;S.insert({x,index});}
    bool have(T x) {
        int rank = S.order_of_key({x, 0});
        if (rank >= static_cast<int>(S.size())) return false;
        auto it = S.find_by_order(rank);
        return it!=S.end()&&it->first == x;
    }
    typename Type::iterator find(T x){
        int rank = S.order_of_key({x, 0});
        if (rank >= static_cast<int>(S.size())) return S.end();
        auto it = S.find_by_order(rank);
        if(it==S.end()||(*it).first!=x)return S.end();
        return it;
    }
    bool erase(T x){
        auto p=find(x);
        if(p!=S.end()){
            S.erase(p);return true;
        }
        return false;
    }
    typename Type::iterator erase(typename Type::iterator it){return S.erase(it);}
    int getrank(T x){return S.order_of_key({x,0});}///返回严格小于 x 的元素个数 (以 Cmp_Fn 作为比较逻辑)
    typename Type::iterator select(int x){return S.find_by_order(x);}///返回 Cmp_Fn 比较的排名所对应元素
    typename Type::iterator prev(T x){///x 严格前驱
        int t=S.order_of_key({x,0});
        if(t==0)return S.end();
        else return select(t-1);
    }
    typename Type::iterator succ(T x){///x 严格后继
        return S.upper_bound({x,MaxN});
    }
    typename Type::iterator lower_bound(T x){return S.lower_bound({x,0});}
    typename Type::iterator upper_bound(T x){return S.upper_bound({x,MaxN});}
};

```

```

typename Type::iterator begin(){return S.begin();}
typename Type::iterator end(){return S.end();}
bool empty(){return S.empty();}
size_t size(){return S.size();}
};

```

虚树

```

int dfn[MAXN];
int h[MAXN], m, a[MAXN], len; // 存储关键点
bool cmp(int x, int y) {
    return dfn[x] < dfn[y]; // 按照 dfs 序排序
}
void build_virtual_tree() {
    sort(h + 1, h + m + 1, cmp); // 把关键点按照 dfs 序排序
    for (int i = 1; i < m; ++i) {
        a[++len] = h[i];
        a[++len] = lca(h[i], h[i + 1]); // 插入 lca
    }
    a[++len] = h[m];
    sort(a + 1, a + len + 1, cmp); // 把所有虚树上的点按照 dfs 序排序
    len = unique(a + 1, a + len + 1) - a - 1; // 去重
    for (int i = 1, lc; i < len; ++i) {
        lc = lca(a[i], a[i + 1]);
        conn(lc, a[i + 1]); // 连边, 如有边权 就是 distance(lc, a[i+1])
    }
}

```

LinkCutTree

```

struct LinkCutTrees{
    #define MaxN 410000
    int Son[MaxN][2], Dad[MaxN];
    int Sum[MaxN], Data[MaxN];
    bool Rev[MaxN];
    void PushUp(int now){
        //Size[now]=Size[Son[now][0]]+Size[Son[now][1]]+1;
        Sum[now]=Sum[Son[now][0]]^Sum[Son[now][1]]^Data[now];
    }
    void PushDown(int now){
        if(Rev[now]){
            if(Son[now][0]){
                Rev[Son[now][0]]^=1;
                std::swap(Son[Son[now][0]][0], Son[Son[now][0]][1]);
            }
            if(Son[now][1]){
                Rev[Son[now][1]]^=1;
                std::swap(Son[Son[now][1]][0], Son[Son[now][1]][1]);
            }
        }
    }
}

```

```

    }
    Rev[now]=false;
}
}
bool NotRoot(int now){return Son[Dad[now]][0]==now||Son[Dad[now]][1]==now;}
bool Which(int now){return Son[Dad[now]][1]==now;}
void Union(int dad,int son){if(son!=0)Dad[son]=dad;}
void Rotate(int now){
    int dad=Dad[now];bool flag=Which(now);
    Son[dad][flag]=Son[now][!flag];
    Union(dad,Son[now][!flag]);
    if(NotRoot(dad))Son[Dad[dad]][Which(dad)]=now;//虚边不能乱接
    Union(Dad[dad],now);
    Son[now][!flag]=dad;
    Union(now,dad);
    PushUp(dad);PushUp(now);
    return;
}
void SplayInit(int now){
    if(NotRoot(now))SplayInit(Dad[now]);
    PushDown(now);
}
void Splay(int now){
    SplayInit(now);
    for(int dad=Dad[now];NotRoot(now);Rotate(now),dad=Dad[now])
        if(NotRoot(dad))Rotate(Which(now)^Which(dad)?now:dad);
}
void Access(int now){
    for(int tmp=0;now;tmp=now,now=Dad[now])//son 比 now 深, 所以 now 的右儿子为 son
        Splay(now),Son[now][1]=tmp,PushUp(now);
}
void MakeRoot(int now){
    Access(now);Splay(now);//now 到根之后, 没有左子树, 深度最深
    Rev[now]^=1;//翻转之后, 没有右子树, 深度最浅, 即为根
    std::swap(Son[now][0],Son[now][1]);
}
int FindRoot(int now){
    Access(now);Splay(now);
    while(Son[now][0])PushDown(now),now=Son[now][0];//找一个深度最小的点就是根
    Splay(now);
    return now;
}
bool Link(int x,int y){
    MakeRoot(x);
    if(FindRoot(y)==x)return false;//在一棵树内
    Dad[x]=y;
    return true;
}
void Split(int x,int y){//把 x-y 拉成一条链
    MakeRoot(x);Access(y);Splay(y);
    //把 y 当成 Splay 根, 然后就可以通过 y 询问链信息
}
bool Cut(int x,int y){

```

```

        MakeRoot(x);
        if(FindRoot(y)!=x||Dad[y]!=x||Son[y][0])return false;
//建议手模
        Dad[y]=0;Son[x][1]=0;
        return true;
    }
    int Query(int x,int y){Split(x,y);return Sum[y];}
    void Modify(int x,int v){Access(x);Splay(x);Data[x]=v;PushUp(x);}
    #undef MaxN
};

```

点分治

```

#include<cstdio>
#include<cstring>
#include<bitset>
using namespace std;
const int MaxN=11000;
const int MaxM=21000;
const int MaxQ=110;
const int MaxK=11000000;
template<typename datatype>
inline datatype Max(const datatype&x,const datatype&y){return(x>y?x:y);}
template<typename datatype>
inline datatype Min(const datatype&x,const datatype&y){return(x<y?x:y);}
int Last[MaxN];int cnt;
int Next[MaxM],To[MaxM];
int Cost[MaxM];
inline void Insert(int from,int to,int cost){
    To[cnt]=to;Cost[cnt]=cost;
    Next[cnt]=Last[from];Last[from]=cnt++;
}
int Size[MaxN],Maxi[MaxN],root,sum;
bool Visit[MaxN];
inline void CalcRoot(int from,int dad){//得到 Size, 找到重心
    Size[from]=1;Maxi[from]=0;
    for(int k=Last[from];k!=-1;k=Next[k]){
        if(To[k]==dad||Visit[To[k]])continue;
        CalcRoot(To[k],from);
        Size[from]+=Size[To[k]];
        Maxi[from]=Max(Maxi[from],Size[To[k]]);
    }
    Maxi[from]=Max(Maxi[from],sum-Size[from]);
    if(Maxi[from]<Maxi[root])root=from;
}
int Tmp[MaxN],Dist[MaxN],maxk;
inline void CalcDist(int from,int dad){
    //收集子树中的信息, 思考过程可以发现是不重不漏的
    if(Dist[from]<=maxk)Tmp[++Tmp[0]]=Dist[from];
    for(int k=Last[from];k!=-1;k=Next[k]){
        if(To[k]==dad||Visit[To[k]])continue;
    }
}

```

```

        Dist[To[k]]=Dist[from]+Cost[k];
        CalcDist(To[k],from);
    }
}
int Sta[MaxN];
int q,Query[MaxQ];
bool Ans[MaxQ];
bitset<MaxK>Test;
inline void CalcAns(int from){
    //统计一手答案
    static int top;top=0;
    for(int k=Last[from];k!=-1;k=Next[k]){
        if(Visit[To[k]])continue;
        Dist[To[k]]=Cost[k];
        Tmp[0]=0;CalcDist(To[k],from);
        for(int i=1;i<=Tmp[0];i++)
            for(int j=1;j<=q;j++)
                if(Query[j]>=Tmp[i])Ans[j]|=Test[Query[j]-Tmp[i]];
        for(int i=1;i<=Tmp[0];i++)
            Sta[++top]=Tmp[i],Test[Tmp[i]]=true;
    }
    for(int i=1;i<=top;i++)
        Test[Sta[i]]=false;
}
inline void Solve(int from){
    Visit[from]=Test[0]=true;CalcAns(from);
    for(int k=Last[from];k!=-1;k=Next[k]){
        if(Visit[To[k]])continue;
        sum=Size[To[k]];//这个写法不会影响时间复杂度
        Maxi[root=0]=0x3f3f3f3f;
        CalcRoot(To[k],from);
        Solve(root);
    }
}
int n;
int main(){
    memset(Last,-1,sizeof(Last));
    scanf("%d%d",&n,&q);
    for(int i=1;i<=n-1;i++){
        int u,v,w;
        scanf("%d%d%d",&u,&v,&w);
        Insert(u,v,w);
        Insert(v,u,w);
    }
    for(int i=1;i<=q;i++)
        scanf("%d",&Query[i]),maxk=Max(maxk,Query[i]);
    Maxi[root]=sum=n;
    CalcRoot(1,0);
    Solve(root);
    for(int i=1;i<=q;i++)
        printf("%s\n",Ans[i]? "AYE" : "NAY");
    return 0;
}

```

线段树合并

```
#include<cstdio>
#include<cstring>
#include<algorithm>
using namespace std;

const int IOSize=(1<<15)+1;
int input=IOSize;
char InPut[IOSize];
inline char GetChar(){
    if(input>=IOSize)fread(InPut,1,IOSize,stdin),input=0;
    return InPut[input++];
}
template<typename datatype>
inline void Read(datatype&s){
    static bool f;f=false;s=0;
    static char c;c=GetChar();
    while(c>'9' || c<'0'){if(c=='-')f=true;c=GetChar();}
    while(!(c>'9' || c<'0')){s=s*10+c-48;c=GetChar();}
    if(f==true)s=-s;
}
template<typename datatype>
inline void Read(datatype&x,datatype&y){
    Read(x);Read(y);
}
template<typename datatype>
inline void Read(datatype&x,datatype&y,datatype&z){
    Read(x);Read(y);Read(z);
}

template<int MaxN>
struct SegmentTrees{
    int Son[MaxN][2];
    long long Size[MaxN];
    int Trash[MaxN],top,cnt;
    inline void Recycle(int&now){
        Size[now]=0;
        Son[now][0]=Son[now][1]=0;
        Trash[++top]=now;
        now=0;
    }
    inline int Create(){return top?Trash[top--]:++cnt;}
    inline void PushUp(int now){
        Size[now]=Size[Son[now][0]]+Size[Son[now][1]];
    }
    inline void Build(int&now,int Left,int Right,long long*Arr){
        if(!now)now=Create();
        if(Left==Right){
            Size[now]=Arr[Left];
            return;
        }
    }
```



```

    }
    else{
        int Mid=(Left+Right)>>1;
        Build(Son[now][0],Left,Mid,Arr);
        Build(Son[now][1],Mid+1,Right,Arr);
        PushUp(now);
    }
}

inline int Merge(int x,int y,int Left,int Right){
    if(!x||!y)return x+y;
    if(Left==Right){
        Size[x]+=Size[y];
        Recycle(y);
        return x;
    }
    else{
        int Mid=(Left+Right)>>1;
        Son[x][0]=Merge(Son[x][0],Son[y][0],Left,Mid);
        Son[x][1]=Merge(Son[x][1],Son[y][1],Mid+1,Right);
        Recycle(y);
        PushUp(x);
        return x;
    }
}

inline void Split(int&x,int&y,int left,int right,int Left,int Right){
    if(x==0)return;
    if(left<=Left&&Right<=right){
        y=x;x=0;
        return ;
    }
    else{
        int Mid=(Left+Right)>>1;
        if(!y)y=Create();
        if(left<=Mid)
            Split(Son[x][0],Son[y][0],left,right,Left,Mid);
        if(Mid+1<=right)
            Split(Son[x][1],Son[y][1],left,right,Mid+1,Right);
        PushUp(x);
        PushUp(y);
    }
}

inline void Update(int&now,int pos,long long val,int Left,int Right){
    if(!now)now=Create();
    if(Left==Right){
        Size[now]+=val;
        return;
    }
    else{
        int Mid=(Left+Right)>>1;
        if(pos<=Mid)Update(Son[now][0],pos,val,Left,Mid);
        else Update(Son[now][1],pos,val,Mid+1,Right);
        PushUp(now);
    }
}

```

```

}
inline int Select(int&now,long long rank,int Left,int Right){
    if(Left==Right)return Left;
    else{
        int Mid=(Left+Right)>>1;
        if(rank<=Size[Son[now][0]])
            return Select(Son[now][0],rank,Left,Mid);
        else
            return Select(Son[now][1],rank-Size[Son[now][0]],Mid+1,Right);
    }
}
inline long long Query(int now,int left,int right,int Left,int Right){
    if(!now)return 0;
    if(left<=Left&&Right<=right)return Size[now];
    else{
        int Mid=(Left+Right)>>1;
        long long tmp1=0,tmp2=0;
        if(left<=Mid)tmp1=Query(Son[now][0],left,right,Left,Mid);
        if(Mid+1<=right)tmp2=Query(Son[now][1],left,right,Mid+1,Right);
        return tmp1+tmp2;
    }
}
};

```

```
SegmentTrees<210000*18+2*210000>T;
```

```

int n,m,tot,Root[210000];
int opt,x,y,p;long long k,ans,Arr[210000];
int main(){
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)
        Read(Arr[i]);
    T.Build(Root[tot=1],1,n,Arr);
    for(int i=1;i<=m;i++){
        Read(opt);
        switch(opt){
            case 0:
                Read(p,x,y);
                T.Split(Root[p],Root[++tot],x,y,1,n);
                break;
            case 1:
                Read(x,y);
                T.Merge(Root[x],Root[y],1,n);
                break;
            case 2:
                Read(p);Read(k);Read(x);
                T.Update(Root[p],x,k,1,n);
                break;
            case 3:
                Read(p,x,y);
                printf("%lld\n",T.Query(Root[p],x,y,1,n));
                break;
            case 4:

```

```

        Read(p);Read(k);
        if(k>T.Size[Root[p]])ans=-1;
        else ans=T.Select(Root[p],k,1,n);
        printf("%lld\n",ans);
        break;
    }
}
return 0;
}

```

李超线段树

```

const double Eps=1e-9;
const int Mod1=39989;
const int Mod2=1000000000;
typedef pair<double,int>Pdi;
inline int Cmp(const double&x,const double&y){
    if(x-y>Eps)return 1;
    if(y-x>Eps)return -1;
    return 0;
}
struct Lines{
    double k,b;
};Lines Pi[110000];
int cnt;
int Si[160000];
inline double Calc(int id,double x){
    return Pi[id].b+x*Pi[id].k;
}
inline void Add(int x0,int y0,int x1,int y1){
    cnt++;
    if(x0==x1)//直线斜率不存在
        Pi[cnt]={0,1.0*max(y0,y1)};
    else
        Pi[cnt]={1.0*(y1-y0)/(x1-x0),y0-1.0*(y1-y0)/(x1-x0)*x0};
}
inline void Update1(int now,int Left,int Right,int id){
    int mid=(Left+Right)>>1;
    if(Cmp(Calc(id,mid),Calc(Si[now],mid))==1)
        swap(Si[now],id);
    int bl=Cmp(Calc(id,Left),Calc(Si[now],Left));
    int br=Cmp(Calc(id,Right),Calc(Si[now],Right));
    if(bl==1||(!bl&&id<Si[now]))Update1(now<<1,Left,mid,id);
    if(br==1||(!br&&id<Si[now]))Update1(now<<1|1,mid+1,Right,id);
}
inline void Update2(int now,int Left,int Right,int left,int right,int id){
    if(left<=Left&&Right<=right){
        Update1(now,Left,Right,id);
        return ;
    }
    else{

```

```

        int mid=(Left+Right)>>1;
        if(left<=mid)Update2(now<<1,Left,mid,left,right,id);
        if(mid+1<=right)Update2(now<<1|1,mid+1,Right,left,right,id);
    }
}
inline Pdi Max(Pdi x,Pdi y){
    if(Cmp(x.first,y.first)==-1)return y;
    else if(Cmp(x.first,y.first)==1)return x;
    else return x.second<y.second?x:y;
}
inline void Query(int now,int Left,int Right,int pos,Pdi&val){
    double res=Calc(Si[now],pos);
    val=Max(val,{res,Si[now]});
    if(Left==Right)return ;
    else{
        int mid=(Left+Right)>>1;
        if(pos<=mid)Query(now<<1,Left,mid,pos,val);
        else Query(now<<1|1,mid+1,Right,pos,val);
    }
}
inline Pdi Query(int now,int Left,int Right,int pos){
    Pdi val={0,0};
    Query(now,Left,Right,pos,val);
    return val;
}
int n,lastans=0;
int main(){
    scanf("%d",&n);
    while(n--){
        int opt;scanf("%d",&opt);
        if(opt==1){
            int x0,y0,x1,y1;
            scanf("%d%d%d%d",&x0,&y0,&x1,&y1);
            x0=(x0+lastans-1+Mod1)%Mod1+1;
            x1=(x1+lastans-1+Mod1)%Mod1+1;
            y0=(y0+lastans-1+Mod2)%Mod2+1;
            y1=(y1+lastans-1+Mod2)%Mod2+1;
            if(x0>x1)swap(x0,x1),swap(y0,y1);
            Add(x0,y0,x1,y1);
            Update2(1,1,Mod1,x0,x1,cnt);
        }
        else{
            int x;scanf("%d",&x);
            x=(x+lastans-1+Mod1)%Mod1+1;
            printf("%d\n",lastans=Query(1,1,Mod1,x).second);
        }
    }
    return 0;
}

```

莫队

```
int size;
struct Opts{
    int l,r,id;
    inline bool operator<(const Opts&o){return l/size==o.l/size?r<o.r:l<o.l;}
}; Opts Q[MaxN];
int Ans[MaxN][2];
int A,B,len,Cnt[MaxN],Arr[MaxN];
inline void Add(int pos){
    A+=Cnt[Arr[pos]];
    Cnt[Arr[pos]]++;
    B+=len;
    len++;
}
inline void Del(int pos){
    Cnt[Arr[pos]]--;
    A-=Cnt[Arr[pos]];
    len--;
    B-=len;
}
int n,m;
int main(){
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)
        scanf("%d",&Arr[i]);
    for(int i=1;i<=m;i++)
        scanf("%d%d",&Q[i].l,&Q[i].r),Q[i].id=i;
    size=n/sqrt(m); //最优块长
    std::sort(Q+1,Q+m+1); //排序询问
    int l=1,r=0; //只要保证是个空区间就行
    for(int i=1;i<=m;i++){
        if(Q[i].l==Q[i].r){
            Ans[Q[i].id][0]=0;
            Ans[Q[i].id][1]=1;
        }
        while(l>Q[i].l)Add(--l);
        while(r<Q[i].r)Add(++r);
        while(l<Q[i].l)Del(l++);
        while(r>Q[i].r)Del(r--); //先加后减保证区间长度不为负数
        int g=Gcd(A,B);
        Ans[Q[i].id][0]=A/g;
        Ans[Q[i].id][1]=B/g;
    }
    for(int i=1;i<=m;i++)
        printf("%d/%d\n",Ans[i][0],Ans[i][1]);
    return 0;
}
```

树上莫队

```
template<typename datatype>
inline void Swap(datatype&x,datatype&y){datatype t=x;x=y;y=t;}
```

```

namespace Chx{
    const int MaxN=110000;
    const int MaxM=210000;
    int Last[MaxN],cm;
    int To[MaxM],Next[MaxM];
    inline void Insert(int from,int to){
        To[cm]=to;
        Next[cm]=Last[from];Last[from]=cm++;
    }
    int Sta[MaxN],top;
    int Bel[MaxN],tot,size;
    int Dad[MaxN][18],Dep[MaxN];
    inline void Dfs(int from){
        int tmp=top;
        for(int k=Last[from];k!=-1;k=Next[k]){
            if(To[k]==Dad[from][0])continue;
            Dep[To[k]]=Dep[from]+1;
            Dad[To[k]][0]=from;
            Dfs(To[k]);
            if(top-tmp>=size){
                ++tot;
                while(top!=tmp)Bel[Sta[top--]]=tot;
            }
        }
        Sta[++top]=from;
    }
    struct Opts{
        int a,b,t,id;
        inline bool operator<(const Opts&o)const{return Bel[a]==Bel[o.a]?(Bel[b]==Bel[o.b]?t<o.t:Bel[b]<Bel
    };Ops Q[MaxN];int qcnt;
    struct Changes{
        int pos,val;
    };Changes C[MaxN];int ccnt;
    bool Visit[MaxN];
    int Num[MaxN],Arr[MaxN];
    long long Ans[MaxN],Vi[MaxN],Wi[MaxN],ans;
    inline void Reverse(int x){
        if(Visit[x])ans-=Vi[Arr[x]]*Wi[Num[Arr[x]]--];
        else ans+=Vi[Arr[x]]*Wi[++Num[Arr[x]]];
        Visit[x]^=true;
    }
    inline int GetLca(int x,int y){
        if(Dep[x]<Dep[y])Swap(x,y);
        for(int i=17;i>=0;i--){
            if(Dep[Dad[x][i]]>=Dep[y])
                x=Dad[x][i];
        }
        if(x==y)return x;
        for(int i=17;i>=0;i--){
            if(Dad[x][i]!=Dad[y][i])
                x=Dad[x][i],y=Dad[y][i];
        }
        return Dad[x][0];
    }
    inline void PathModify(int x,int y){

```

```

    if(Dep[x]<Dep[y])Swap(x,y);
    while(Dep[x]>Dep[y]){
        Reverse(x);
        x=Dad[x][0];
    }
    while(x!=y){
        Reverse(x);
        Reverse(y);
        x=Dad[x][0];
        y=Dad[y][0];
    }
}
inline void Modify(int x){
    if(Visit[C[x].pos]){
        Reverse(C[x].pos);
        Swap(Arr[C[x].pos],C[x].val);
        Reverse(C[x].pos);
    }
    else Swap(Arr[C[x].pos],C[x].val);
}

int n,m,q;
inline void Main(){
    memset(Last,-1,sizeof(Last));
    scanf("%d%d%d",&n,&m,&q);
    for(int i=1;i<=m;i++)scanf("%lld",&Vi[i]);
    for(int j=1;j<=n;j++)scanf("%lld",&Wi[j]);
    for(int i=1;i<=n-1;i++){
        int x,y;scanf("%d%d",&x,&y);
        Insert(x,y);Insert(y,x);
    }
    for(int i=1;i<=n;i++)scanf("%d",&Arr[i]);
    for(int i=1;i<=q;i++){
        int opt,x,y;scanf("%d%d%d",&opt,&x,&y);
        if(opt==0){
            ccnt++;
            C[ccnt]={x,y};
        }
        else{
            qcnt++;
            Q[qcnt]={x,y,ccnt,qcnt};
        }
    }
    size=ceil(pow(n,2.0/3.0));
    Dep[1]=1;
    Dfs(1);
    if(top){
        tot++;
        while(top)Bel[Sta[top--]]=tot;
    }
    for(int i=1;i<=17;i++)
        for(int j=1;j<=n;j++)
            Dad[j][i]=Dad[Dad[j][i-1]][i-1];
}

```

```

        sort(Q+1,Q+qcnt+1);
        int a=1,b=1,now=0;
        for(int i=1;i<=qcnt;i++){
            PathModify(a,Q[i].a);
            PathModify(b,Q[i].b);
            a=Q[i].a;
            b=Q[i].b;
            while(now<Q[i].t)Modify(++now);
            while(now>Q[i].t)Modify(now--);
            int lca=GetLca(Q[i].a,Q[i].b);
            Reverse(lca);
            Ans[Q[i].id]=ans;
            Reverse(lca);
        }
        for(int i=1;i<=qcnt;i++)
            printf("%lld\n",Ans[i]);
    }
}
int main(){
    Chx::Main();
    return 0;
}

```

帶修莫隊

```

const int MaxN=200000;
int size;
struct Opts{
    int l,r,t,id;
    inline bool operator<(const Opts&o){
        return l/size==o.l/size?(r/size==o.r/size?t<o.t:r<o.r):l<o.l;
    }
}; Opts Q[MaxN]; int qcnt;
struct Changes{
    int pos,val;
}; Changes C[MaxN]; int ccnt;
int Arr[MaxN];
const int MaxS=1100000;
int Num[MaxS],ans;
inline void Add(int c){
    if(Num[c]==0)ans++;
    Num[c]++;
}
inline void Del(int c){
    Num[c]--;
    if(Num[c]==0)ans--;
}
inline void Modify(int i,int j){
    if(Q[i].l<=C[j].pos&&C[j].pos<=Q[i].r){
        Del(Arr[C[j].pos]);
        Add(C[j].val);
    }
    std::swap(Arr[C[j].pos],C[j].val); //下次修改一定是撤回
}

```



```

}
char O[10];int n,m,Ans[MaxN];
int main(){
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)
        scanf("%d",&Arr[i]);
    for(int i=1;i<=m;i++){
        scanf("%s",O);
        if(O[0]=='R'){
            int x,y;scanf("%d%d",&x,&y);++ccnt;
            C[ccnt]={x,y};
        }
        else{
            int x,y;scanf("%d%d",&x,&y);++qcnt;
            Q[qcnt]={x,y,ccnt,qcnt};
        }
    }
    size=pow(n,2.0/3.0);//最优块长
    std::sort(Q+1,Q+qcnt+1);
    int l=2,r=1,now=0;
    for(int i=1;i<=qcnt;i++){
        while(l>Q[i].l)Add(Arr[--l]);
        while(r<Q[i].r)Add(Arr[++r]);
        while(l<Q[i].l)Del(Arr[l++]);
        while(r>Q[i].r)Del(Arr[r--]);
        while(now<Q[i].t)Modify(i,++now);//看时间线判断是修改还是撤回
        while(now>Q[i].t)Modify(i,now--);
        Ans[Q[i].id]=ans;
    }
    for(int i=1;i<=qcnt;i++)
        printf("%d\n",Ans[i]);
    return 0;
}

```

只增莫队

```

const int MaxN=110000;
int size;
struct Querys{
    int l,r,id;
    inline bool operator<(const Querys&q)const{
        return l/size==q.l/size?r<q.r:l<q.l;
    }
};Querys Q[MaxN];
int Arr[MaxN],Num[MaxN],Copy[MaxN],tot;
long long ans,tmp,Ans[MaxN];
inline void Add(int t){
    Num[t]++;
    ans=std::max(ans,1ll*Num[t]*Copy[t]);
}
int n,m;
int main(){
    scanf("%d%d",&n,&m);

```

```

for(int i=1;i<=n;i++)
    scanf("%d",&Arr[i]),Copy[++tot]=Arr[i];
std::sort(Copy+1,Copy+tot+1);
tot=std::unique(Copy+1,Copy+tot+1)-Copy-1;
for(int i=1;i<=n;i++)
    Arr[i]=std::lower_bound(Copy+1,Copy+tot+1,Arr[i])-Copy;
size=n/sqrt(m)+1;
for(int i=1;i<=m;i++){
    scanf("%d%d",&Q[i].l,&Q[i].r);
    Q[i].id=i;
    if(Q[i].l/size==Q[i].r/size){//同块暴力
        for(int j=Q[i].l;j<=Q[i].r;j++){
            ++Num[Arr[j]];
            Ans[i]=std::max(Ans[i],1ll*Num[Arr[j]]*Copy[Arr[j]]);
        }
        for(int j=Q[i].l;j<=Q[i].r;j++)--Num[Arr[j]];
    }
}
std::sort(Q+1,Q+m+1);
int bl=-1,l,r;
for(int i=1;i<=m;i++){
    if(Q[i].l/size==Q[i].r/size)continue;
    if(Q[i].l/size!=bl){//换块归位
        ans=0;
        bl=Q[i].l/size;
        memset(Num,0,sizeof(Num));
        l=(bl+1)*size;
        r=(bl+1)*size-1;//右端点递增
    }
    while(r<Q[i].r)Add(Arr[++r]);//右端点递增
    tmp=ans;//记录以备撤销
    while(l>Q[i].l)Add(Arr[--l]);//左端点增加
    Ans[Q[i].id]=ans;
    while(l<(bl+1)*size)--Num[Arr[l++]);//回滚
    ans=tmp;//撤销
}
for(int i=1;i<=m;i++)
    printf("%lld\n",Ans[i]);
return 0;
}

```